3. Sorozattal reprezentált típusok

Olyan típusok definiálása, majd osztály diagrammal történő leírása, ahol a reprezentációhoz elemek sorozatára van szükségünk.

1. Asszociatív tömb
   Valósítsuk meg az asszociatív tömb típust típusát, amely olyan kulcs-adat párokat tároló gyűjteményt jellemez, amelyben kulcs alapján lehet visszakeresni az értékeket. A kulcs egy egész szám legyen, az adat pedig szöveg. Fontos, hogy a kulcs egyedi legyen.

Típusdefiníció:

Típus értékek:	**Típus műveletek:**
Map asszociatív tömbök (speciális tárolók) halmaza. Egy tároló elemei $\mathbb{Z} \times \mathbb{S}$ (kulcs-adat) típusú párok, és egy ilyen elem a kulcsa alapján egyértelműen beazonosítható	**map := SetEmpty(map)** map : Map *//kiüríti az asszociatív tömböt* **c := Count(map)** map : Map, c : $\mathbb{N}$ *//megadja az elemek számát* **map := Insert(map,e)** map : Map, e : $\mathbb{Z} \times \mathbb{S}$ *//új elemet tesz be, ha a kulcsa még nem létezik* **map := Erase(map, key)** map : Map, key : $\mathbb{Z}$ *// törli az adott kulcsú elemet, ha a kulcs létezi, különben hiba* **l := In(map, key)** map : Map, key : $\mathbb{Z}$, l : $\mathbb{L}$ *//lekérdezi, van-e adott kulcsú elem* **data := map[key]** map : Map, key : $\mathbb{Z}$, data : $\mathbb{S}$ *// lekérdezi az adott kulcsú elem adatát, ha a kulcs létezik, különben hiba*

[1.kvíz]

A kulcs-adat párok tárolásához egy **rendezett sorozatot** fogunk használni. A sorozatban kulcs szerint szigorúan monoton növekvő sorrendben tároljuk az elemeket. Ekkor a kulcsok kereséséhez használhatjuk a logaritmikus keresést. Ha tömbben tárolnánk az elemeket, akkor beszúrásnál „helyet kell csinálni” az új elemnek: hátrébb kell csúsztatni az új elemnél nagyobb kulcsúakat; törlésnél pedig a keletkezett „lyukat” el kell tüntetni: előrébb kell csúsztatni a törlés pozíciója utáni elemeket.

(Megjegyzés: található hatékonyabb tárolás (bináris keresőfák, hash tábla), lásd majd Algoritmusok tárgyban.)

Típus reprezentáció:

seq: Element* – az elemeket kulcsuk szerint rendezetten tároló sorozat,

ahol

Element = rec(key: $\mathbb{Z}$, data: $\mathbb{S}$)

Típusműveletek implementációja:

SetEmpty(map)

seq := <>

c := Count(map)

c := |seq|

map := Insert(map,e) l : $\mathbb{L}$, ind : $\mathbb{N}$

l, ind := logSearch(seq, e.key)	
¬l	
seq := seq[1 .. ind-1] ⊕ <e> ⊕ seq[ind .. |seq|]	—

map := Erase(map, key) l : $\mathbb{L}$, ind : $\mathbb{N}$ [2.kvíz]

l, ind := logSearch(seq, key)	
l	
seq := seq[1 .. ind-1] ⊕ seq[ind+1 .. |seq|]	**Hiba**: nem létező kulcs

l := In(map, key)

l, ind := logSearch(seq, key)	ind : $\mathbb{N}$

data := map[key]

l, ind := logSearch(seq, key)	
l	
data := seq[ind].data	**Hiba**: nem létező kulcs

Több művelet egy kulcs szerinti kereséssel indul, amelyhez a logaritmikus keresés algoritmusát használjuk. Ez egy fontos privát algoritmusa lesz a típusunknak.

Egy lépéssel még kiegészítjük az algoritmust: ha nincs benne a keresett kulcs a tömbben, akkor az első nála nagyobb elem sorszámát adja meg. Ezzel ügyesebb lesz az insert művelet: nem kulcs-összehasonlításra támaszkodik az eltolás ciklusa, hiszen tudjuk, hogy mettől meddig kell a tömb elemeit jobbra tolni. Ha nincs a keresett kulcsnál nagyobb a tömbben, akkor az utolsó elem utáni sorszámot kapjuk vissza.

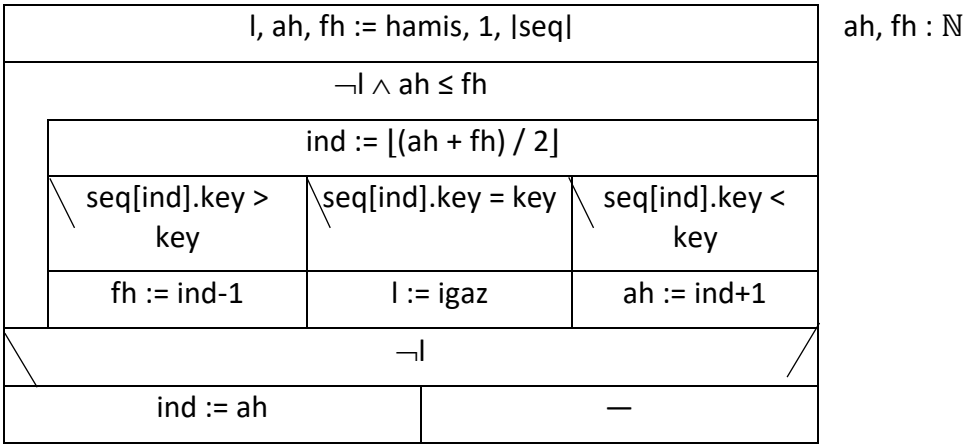
Kulcs szerinti logaritmikus keresés [nem kell rá túl sok időt szánni, hiszen a logaritmikus keresés az előző gyakorlaton szerepelt]

A = (seq : Element*, key : $\mathbb{Z}$, l : $\mathbb{L}$, ind : $\mathbb{N}$)

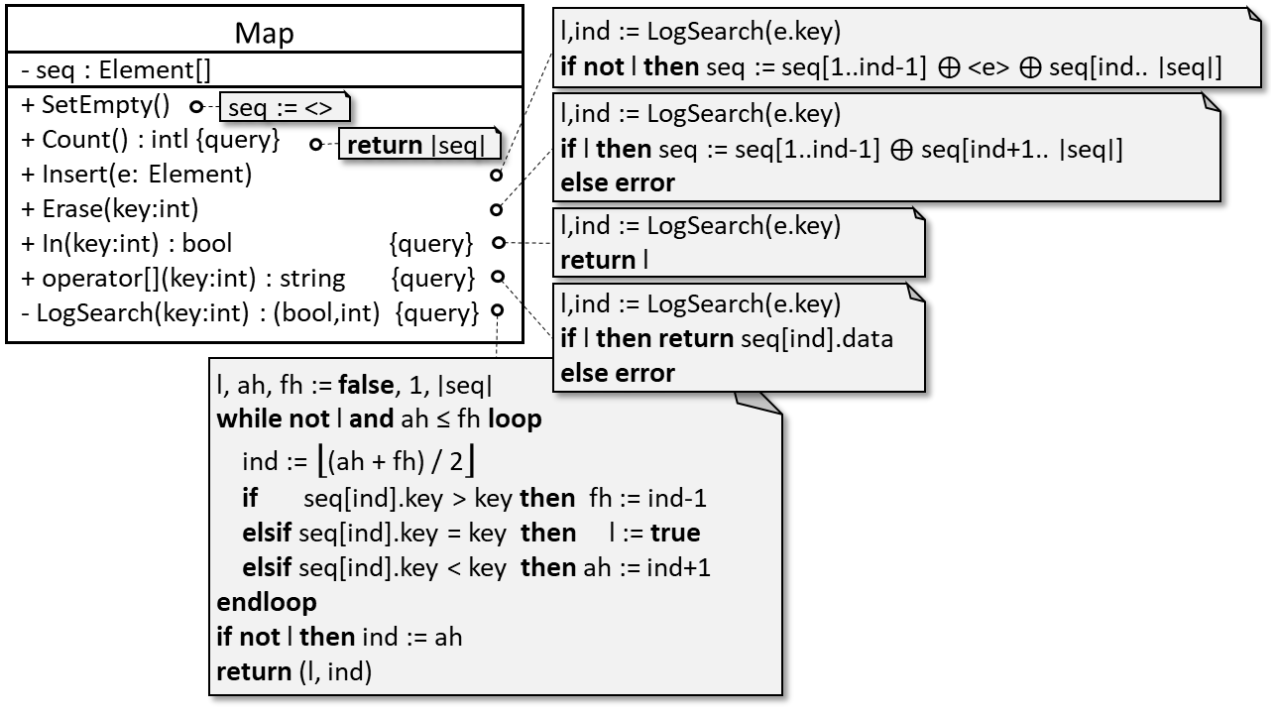
Ef = (seq=seq_0 $\wedge$ key=key_0 $\wedge$ $\forall i \in [1 .. |seq|-1] : seq[i].key < seq[i+1].key$)

Uf=(Ef $\wedge$ l = $\exists i \in [1 .. |seq|] : seq[i].key = key \wedge$
(l $\rightarrow$ ind$\in$[1 .. |seq|] $\wedge$ seq[ind].key = key) $\wedge$
($\neg$l $\rightarrow$ $\forall i \in [1..ind-1]: seq[i].key < key \wedge \forall i \in [ind.. |seq|] : seq[i].key > key$))

l, ind := logSearch (seq, key)



Osztály diagram:



2. Prioritásos sor

Készítsünk maximum prioritásos sor típust. Ennek elemei két mezőből állnak (prioritás (egész), adat (szöveg)). A sorból mindig a legnagyobb prioritású elemet vesszük ki (több legnagyobb esetén nem meghatározott, hogy melyiket).

Ebben a félévben Algoritmusok és adatszerkezetek tárgyból fogják tanulni a kupaccal ábrázolt prioritásos sort. Annak felülete az alábbi:
http://aszt.inf.elte.hu/~asvanyi/ad/ad1jegyzet.pdf

Mivel a tárgyakat párhuzamosan tanulják, érdemes ugyanazokat az elnevezéseket használni. Továbbá ismerik már a műveletigény jelölésére az $O\ \Theta\ \Omega$ *aszimptotikus korlátokat.*

Típus értékek:	**Típus műveletek:**
PrQueue a maximum prioritásos sorok halmaza, amely soroknak az elemei $\mathbb{Z} \times \mathbb{S}$ típusú párok.	**SetEmpty(pq)** pq : PrQueue *// Kiüríti a pr sort* **l := isEmpty(pq)** pq : PrQueue, l : $\mathbb{L}$ *//Igazat ad, ha üres a pr sor, hamisat ha nem.* **pq := Add(pq, e)** pq : PrQueue, e : $\mathbb{Z} \times \mathbb{S}$ *//Betesz egy új elemet a pr sorba.* **e := GetMax(pq)** pq : PrQueue, e : $\mathbb{Z} \times \mathbb{S}$ *//Visszadja az egyik legnagyobb prioritású elemet, nem veszi ki. Fontos, hogy a sor nem lehet üres.* **pq, e := RemMax(pq)** pq : PrQueue, e : $\mathbb{Z} \times \mathbb{S}$ *//Kiveszi az egyik legnagyobb prioritású elemet. Fontos, hogy a sor nem lehet üres.*

A műveleteket objektum-orientált stílusban is megadhatjuk:

pq.SetEmpty()

l := pq.isEmpty() {query}

pq.Add(e)

e:= pq.GetMax() {query}

e := pq.RemMax()

Megvalósításnál két módszer között választhatunk. Mindkettőhöz szükségünk van egy sorozatra, de a műveletek futási ideje eltérő lehet.

(1) **Rendezetlen sorozatot** használunk. Az elemeket folyamatosan helyezzük el a sorozatban. Nem tudjuk, melyik közülük a legnagyobb prioritású.
   a. **SetEmpty** : üres sorozatot készít. $\Theta(1)$ (Habár nem tudjuk, hogy a vector clear() metódusa mit is csinál pontosan.)
   b. **isEmpty:** hosszból azonnal eldönthető. $\Theta(1)$
   c. **Add:** az új elemet a sorozat végéhez fűzzük. $\Theta(1)$
   d. **GetMax:** ha nem üres s sor, a tanult maximum kiválasztás algoritmussal megkeressük az egyik legnagyobb prioritású elemet, és visszaadjuk. A sorozat nem változik. $\Theta(n)$
   e. **RemMax:** ha nem üres a sorozat, a tanult maximum kiválasztás algoritmussal megkeressük az egyik legnagyobb prioritású elemet, és kivesszük. $\Theta(n)$

(2) **Rendezett sorozatot** használunk. A sorozatban az elemek prioritás szerint növekvő a sorrendben vannak.
   a. **SetEmpty** : üres sorozatot készít. $\Theta(1)$ (Habár nem tudjuk, hogy a vector clear() metódusa mit is csinál pontosan.)
   b. **isEmpty:** hosszból azonnal eldönthető $\Theta(1)$
   c. **Add:** az új elemet betesszük a rendezettség szerinti helyére, amelyet lineáris kereséssel vagy kiválasztással kell megkeresnünk $O(n)$
   d. **GetMax:** ha nem üres a sorozat, akkor a rendezettség miatt a sorozat utolsó eleme az egyik legnagyobb prioritású elem. A sorozat nem változik. $\Theta(1)$
   e. **RemMax:** ha nem üres a sorozat, akkor a rendezettség miatt a sorozat utolsó eleme az egyik legnagyobb prioritású elem. Azt ki is kell vennünk a sorozatból. $\Theta(1)$

Látjuk, hogy a prioritásos sor ábrázolásánál nem tudjuk a konstans műveletigényt tartani mindegyik művelet esetén. Mindkét módszernél legalább az egyik művelet „lineáris" műveletigényű, azaz az elemek számával arányos. Van egy pici különbség a két lineáris igényű művelet között: míg a maximum kiválasztás minden esetben megvizsgálja az összes elemet, addig a rendezett részbe való beszúráshoz egy keresés kell, amely korábban is leállhat. Ha C++ nyelven a vectort használjuk, akkor az erase(), illetve az insert() műveletekkel tetszés szerint változtathatjuk a sorozatot. Azt, hogy melyik a jobb, annak függvényében lehet eldönteni, hogy melyik művelet lesz gyakoribb, és azt megvalósítani hatékonyan. (*Megjegyzés: Tovább növelhető a hatékonyság kupac adatszerkezettel, erről majd az Algoritmusok és adatszerkezetek tárgyban lesz szó.)*

Ha egy előre megadott méretű tömböt használnánk, amelynél külön megadjuk, hogy az első hány eleme van kitöltve, akkor a beszúrás néhány elem hátra csúsztatását igényli, amely legrosszabb esetben az összes elemet odébb csúsztatja. A RemMax() tehát rendezett esetben is $O(n)$. Igaz, hogy átlagosan az elemek felét érinti majd a hátrébb csúsztatás, tehát itt várhatóan fele annyi lesz az átlagos lépésszám, viszont az elemek hátrébb csúsztatása jóval költségesebb, mintha csak kiolvassuk őket, amelyet a RemMax() rendezetlen esetben csinál. Ez az érv a rendezetlen ábrázolás mellett szól.

Majd a következő feladatban (asszociatív tömb) a rendezett ábrázolást használjuk, ezért itt most a **rendezetlen tömbbel való ábrázolást választjuk**.

Típus reprezentáció:

seq: Element*

- a prioritásos sor elemeit rendezetlenül tároló sorozat, ahol Element = rec(pr : $\mathbb{Z}$, data : $\mathbb{S}$)

Típusműveletek implementációja:

SetEmpty(pq))

seq := <>	(C++ vector: clear())

l := isEmpty(pq)) [3.kvíz]

l := |seq| =0	(C++ vector: size())

pq := Add(pq, e)

seq := seq ⊕ <e>	(C++ vector: push_back())

e := GetMax(pq)

|seq|>0	
max, ind := $MAX_{i=1..[seq|}$(seq[i].pr)	**Hiba:** üres prioritásos sor
e := seq[ind]	

pq, e := RemMax(pq) [4.kvíz]

|seq|>0	
max, ind := $MAX_{i=1..[seq|}$(seq[i].pr)	**Hiba** üres prioritásos sor
e := seq[ind]	
seq[ind] := seq[|seq|]	
pop_back(seq)	

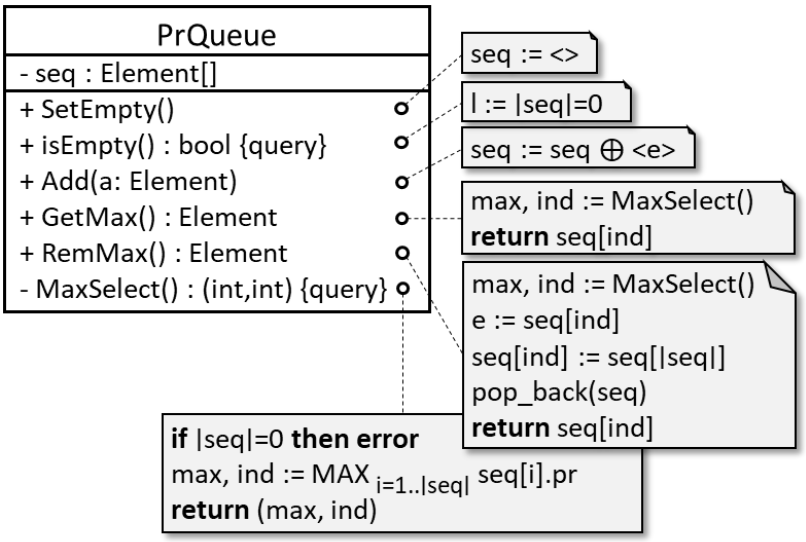
A max, ind := $MAX_{i=1..[seq|}$(seq[i].pr) segédművelet specifikációja és algoritmusa: [5.kvíz]

A = (seq:Element*, e:Element)
Ef = (seq=seq' $\wedge$ |seq|>0)
Uf = (*Ef* $\wedge$ e = seq[ind] $\wedge$
(max, ind) = $MAX_{i=1\ ..\ |seq|}$ seq[i].pr)
ahol max:$\mathbb{Z}$ és ind:$\mathbb{N}$

max, ind := $MAX_{i=1..[seq|}$(seq[i].pr)

max, ind := seq[1].pr, 1	
i = 2 .. |seq|	
max < seq[ind].pr	
max, ind := seq[ind].pr, i	—

Osztály diagram:



Egy feladat megoldása prioritásos sorral:

Egy programozási versenyen csapatok indultak. Ismerjük a csapatok nevét, és a versenyen elért pontszámukat. Készítsünk listát a csapatok eredményéről csökkenő sorrendben. (Feltehető, hogy a csapatok neve egyedi.)

A = (t : Element^n, cout : Item*)

Ef = (t = t_0)

Uf = (t = $t_0 \wedge$ cout = (t elemeit tartalmazza, pr értékei szerint monoton csökkenően felsorolva))

Az utófeltételt végrehajtható formában adjuk meg: először a tömb elemeit bepakolása a pr prioritásos sorba (ezt a C++ kódban lehet közvetlenül a fájlból végezni), majd a sor elemeinek (prioritás szerint csökkenő sorrendben történő) kiírása a konzolra.

Uf=(t = t_0 $\wedge$ pq : PrQueue $\wedge$ pq = $\bigcup_{i=1..n} \{t[i]\} \wedge$ cout = $\bigoplus^{\neg \text{isEmpty}() \text{pq}}$ RemMax(pq))

Mindkét lépés egy összegzés. Az unió művelet a prioritásos sorba tesz be egy új elemet (azaz szemantikailag megegyezik az add() művelettel) úgy, mintha a prioritásos sorhoz unióznánk az új elemet tartalmazó prioritásos sort. Ennek a műveletnek a neutrális eleme az üres prioritásos sor. Az összefűzés addig tart, amíg a pq prioritásos sor nem lesz üres. (Ez már előre vetíti a felsorolós változatát az összegzésnek.)

pq.SetEmpty()
i = 1 .. n
pq.Add(t[i])
cout := <>
¬pq.isEmpty()
cout := cout ⊕ pq.RemMax()

Prioritásos sor megvalósításának tesztelése:

Mivel a gyakorlat egyik témaköre a tesztelés megtervezése, így, tekintsük át, mit, hogyan kellene tesztelni. A tényleges tesztelés megvalósítása a géptermi gyakorlaton történik, lásd a csatolt C++ implementációban.

Vegyük sorra, milyen tesztelést képzelnének el az egyes metódusokhoz (példákat a forráskódban lehet látni):

- SetEmpty() (végrehajtása után az isEmpty() igazat ad)
- isEmpty() (üres / nem üres állapotra kipróbáljuk)
- Add(Item e) (egymás után berakunk elemeket, majd ellenőrizzük az elhelyezésüket)
- MaxSelect() (max és az ind vizsgálandó)
- GetMax() (a maxsearch()-höz képest még a hibás esetet kell tesztelni)
- RemMax() (a max()-hoz képest még a tömb átrendeződését is ellenőrizzük)

remMax() tesztelése			
teszteset	**teszt tömb**	**eredmény**	**tömb új tartalma**
üres intervallum	<>	hiba (kivétel dobás)	<>
egy elemű	<3>	3	<>
több elemű esetek:			
első a legnagyobb	<5,2,3>	5	<3,2>
utolsó a legnagyobb	<1,2,3>	3	<1,2>
belső a legnagyobb	<1,3,2>	3	<1,2>
nem egyértelmű, első és utolsó a legnagyobb	<5,2,5'>	5	<5',2> (az adat rész segítségével ellenőrizhető, hogy az elsőt vettük ki)
nem egyértelmű, belső és utolsó a legnagyobb	<1,3,3'>	3	<1,3'>
mind egyforma	<3,3',3''>	3	<3'',3'>
több egymás utáni remMax(), majd add hatása	<2,3,1>	3 2 1 3 2 1	<2,1> remMax() <1> remMax() <> add(3) add(2) add(1) remMax() <1,2> remMax() <1> remMax() <>